Методы и принципы проектирования современных защищённых систем

Лекция 2

Security Gate релиз-кандидата

SAST · DAST · секреты · SBOM · зависимости · контейнер · 65 минут

Задача

Release candidate 2.0
C-приложение и Docker-образ

→

Решение
RELEASE или BLOCK
с доказательствами

Сканер даёт finding; ответственность за решение остаётся у команды

65 минут · восемь этапов

05 · Подготовка

стенд и репозиторий

05 · Сборка

тест и проблемная ветка

12 · SAST

путь двойного `free` и отчёт SARIF

03 · DAST

сборка с ASAN и выход за границу

10 · Секреты

файл и история Git

12 · Поставка

SBOM и CVE

09 · Контейнер

конфигурация и права

09 · Исправление

код, границы, секреты, образ и Gate

Одна готовая Ubuntu-среда

Хост

```
./lab.sh start
./lab.sh shell
```

→

Ubuntu 24.04

Clang + ASAN · Git · Gitleaks

Syft · Grype · Trivy

`/work/repository` · `/work/reports`

После входа все команды выполняются в одном терминале

Ловушка зелёного теста

```
100% tests passed, 0 tests failed
session closed
```

> Функциональный тест зелёный. Достаточно ли этого для допуска релиза?

Почему Clang Static Analyzer

Бесплатный

open source и доступен локально

Семантический

отслеживает состояния памяти по веткам

Объяснимый

строит трассу от `malloc` до второго `free`



Контрольная точка 1 · анализ пути владения памятью

SARIF: отчёт SAST для машин

`runs → tool`

анализатор `clang` и список правил `unix.Malloc`

`results[]`

`ruleId` · `level` · `message` · `locations` с файлом и строкой

`codeFlows`

трасса: выделение, освобождение, повтор; `importance` шага

Один JSON-формат (OASIS, 2.1.0): отчёты разных анализаторов собираются в одном месте

DAST: запускаем программу под ASAN

Сборка
`clang -fsanitize=address`

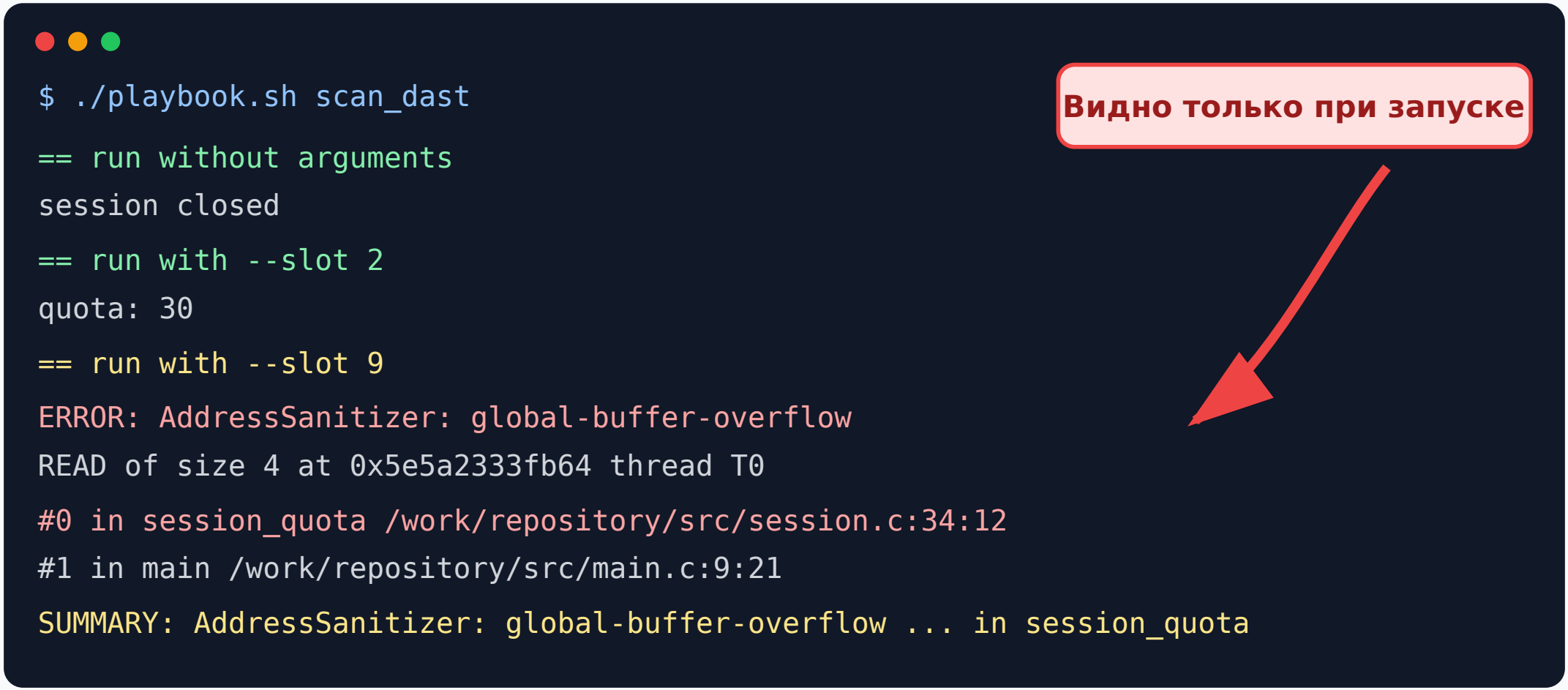
→

Три запуска
без аргументов
`--slot 2` · `--slot 9`

→

Отчёт ASAN
вид ошибки
функция и строка

ASAN проверяет память во время работы; LeakSanitizer при выходе печатает утечки



Контрольная точка 2 · динамический анализ видит только выполненные пути

SAST и DAST видят разное

	SAST	DAST
Читает	исходный код, не запуская	работающую программу
Нашёл	двойной `free`, `session.c:27`	`quota[9]`, `session.c:34`
Не увидел	индекс из командной строки	`--reject`: путь не запускали
Исправление	одна точка `free`	проверка границ `slot`

> Ни один метод не заменяет другой: SAST видит невыполненный код, DAST — реальные данные

Правило владения

До
ветка ошибки: `free`
общий конец: `free`

→

После
обе ветки переходят
в единый `cleanup`

> `free(NULL)` скрывает симптом; единая точка очистки исправляет модель

```
$ ./playbook.sh scan_secrets
2 commits scanned.
leaks found: 2

RuleID: training-api-token
File: config/app.env
Commit: prepare release 2.0

RuleID: training-api-token
File: config/legacy.env
Commit: bootstrap release candidate
```



Контрольная точка 3 · удалить файл недостаточно

Удалить файл ≠ устранить утечку

Действие	Что исправляет	Что остаётся
Удалить строку	текущий файл	токен действует
Удалить файл	рабочее дерево	коммит доступен
Переписать Git	история репозитория	скомпрометированный токен
Ротация	доступ по старому токену	нужно доставить новый

> Сначала отзыв и замена секрета, затем очистка и координация истории

SBOM и CVE — разные артефакты

Syft → SBOM
имя · версия · PURL
состав конкретного образа

→

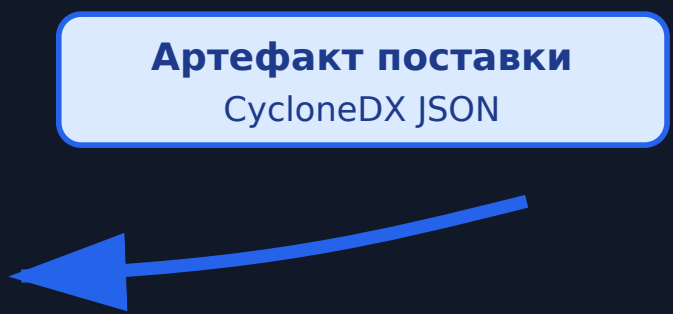
Grype → findings
advisory · severity · fix
состояние базы CVE

SBOM сохраняют с релизом; уязвимости по нему перепроверяют позже

```
$ ./playbook.sh make_sbom
✔ Loaded image practice-2-release:student
✔ Cataloged contents
SBOM: reports/sbom.cdx.json

$ ./playbook.sh scan_container
Dockerfile: Tests 21 · Failures 2 HIGH
AVD-DS-0002 · no non-root USER
AVD-DS-0029 · no --no-install-recommends

Decision: BLOCK
```



Контрольная точка 4 · состав образа и конфигурация контейнера

Поверхность исходного образа

Build = runtime

компилятор и CMake остаются в поставке

Root

процесс получает лишние полномочия

`privileged`

изоляция контейнера ослаблена

Bind mount

исходники доступны на запись

Capabilities

не ограничены явно

Filesystem

доступна запись повсюду

Эталон hardening

```
FROM debian:12-slim AS build
RUN apt-get install -y --no-install-recommends build-essential cmake
# сборка и тесты

FROM debian:12-slim
RUN useradd --system --gid app --create-home user
COPY --from=build /src/build/release-agent ./release-agent
USER user
HEALTHCHECK --interval=30s --timeout=3s CMD ["/app/release-agent"]
ENTRYPOINT ["/app/release-agent"]
```

> Multi-stage удаляет build toolchain из runtime и сокращает SBOM

Security Gate

SAST	DAST	Secrets	SBOM	CVE	Config
0 blocker	0 ошибок ASAN	0 finding	создан	решение	0 HIGH

PASS открывает выпуск

```
$ ./playbook.sh verify_release

No analyzer findings

3 commits scanned.

no leaks found

SBOM generated · CycloneDX

Trivy: 0 HIGH · 0 CRITICAL


[PASS] Security Gate: релиз-кандидат принят

[INFO] компонентов в SBOM: 89
```



Контрольная точка 5 · повторная проверка после исправлений

До и после

Контроль	До	После	Причина
SAST	1 blocker	0	единый cleanup
DAST	1 ошибка ASAN	0	проверка границ
Gitleaks	2	0	ротация и история
SBOM	≈234	≈89	multi-stage
Grype	3 Critical · 25 High	1 Critical · 8 High	меньше пакетов в runtime
Trivy	2 HIGH	0 HIGH	non-root и минимум пакетов

Числа CVE меняются; состав, исправления и решение должны быть воспроизводимы

Итоговый вывод

Безопасный релиз — это доказуемая цепочка контролей от исходного кода до конфигурации запуска.

Найти → объяснить → исправить → повторно проверить → принять решение